

Sustcore

面向 RISC-V / LoongArch64 的 Capability-based 混合内核

Sustcore Team • 2026-06

github.com/sustcore-team/sustcore

目录

- 01 项目概述 — 背景、核心理念与技术特点
- 02 核心创新点 — Capability 安全模型、Linux 子系统兼容、等待线程计数器、从零构建
- 03 系统架构 — 总体分层架构、混合内核设计哲学、自研技术栈
- 04 子系统详解 — 内存管理、VFS、进程/线程、调度器、Endpoint/RPC、设备驱动、构建系统
- 05 测试与验证 — 28 个测试模块、SLUB 调试、QEMU stvec Bug 溯源
- 06 总结与展望 — 待完善之处、中长期规划、致谢

项目概述

Sustcore 是面向 **RISC-V 64** 与 **LoongArch64** 的 **Capability-based 混合内核**，灵感源于著名的 seL4 微内核，但从零开始用 C++ 设计实现。

核心理念

将 VFS、内存管理、进程管理及关键驱动收归内核态，融合 **微内核** 的安全性与模块化，与 **宏内核** 的效率与易用性。仅使用 libfdt 与少量头文件作为第三方库。

技术特点

C++ 实现，支持 C++26 反射。自实现了 ext4 文件系统、ELF 加载器、SLUB/Buddy 分配器、virtio 驱动、PLIC 中断控制器等全部核心组件。

- ▶ 网络协议栈、文件系统、部分驱动等置于用户态运行（规划中），以增强隔离性
- ▶ 不基于任何现有内核，大部分子系统均为原创设计并实现

核心创新点

● Capability 安全模型

双层 CSpace 架构 · Payload + Permission 分离 · CLONE/MIGRATE/MIGRATE-ONCE 三种权限传递 · 引用计数生命周期管理 · 系统调用首个参数均为 CapIdx (类 OOP 设计)

● Linux 子系统兼容

用户态兼容层, 与用户程序共享地址空间。系统调用以 0xFFFF 前缀自然区分, 透明转发到 Linux Subsystem 处理, 无需修改 musl/glibc

● 等待线程计数器

基于等待链的公平调度优化: 被更多线程等待的服务线程获得更多调度权重。RR 时间片加权、CFS vruntime 加权, 等待链深度 d 在微内核中近似 $O(1)$

● 从零构建技术栈

自实现 ext4 文件系统、SLUB 分配器、ELF 加载器、virtio 驱动、6 级调度器等。同时支持 RISC-V 64 与 LoongArch64 双架构, 配套组件化多平台构建系统

Capability 安全模型

双层 CSpace 架构

第一层: CGroup[0..4095] 指针数组;

第二层: 每个 CGroup 含 256 个 Capability 槽位。最大容量 1,048,576。典型进程内存占用仅~36 KiB

CapIdx 编码 (32-bit)

bit[0:7] — CGroup 索引;

bit[8:19] — Capability 槽位;

其余保留位用于安全校验, 防止用户态伪造指针

权限传递机制

- | | |
|------------------|--|
| CAP_CLONE | 创建一个指向同一 Payload、具有相同权限的新 Capability, 原 Capability 保留 (可多次克隆) |
| CAP_MIGRATE | 创建一个指向同一 Payload 的新 Capability, 同时将原 Capability 从 CSpace 中移除 (所有权转移) |
| CAP_MIGRATE_ONCE | 同 MIGRATE, 但新 Capability 的 MIGRATE_ONCE 权限会被移除, 即新 Capability 只能被迁移这一次 |

设计优势: 资源不会凭空“可见”, 必须通过显式的 Capability 传递链到达目标进程。优先级: CLONE > MIGRATE > MIGRATE_ONCE。

Linux 子系统兼容设计

问题背景: 内核系统调用均以 CapIdx 作为首个参数（类 OOP 设计），与 Linux 系统调用 ABI 完全不兼容。比赛要求运行官方给定的二进制程序，无法重写 musl/glibc。

架构设计

Linux Subsystem 被加载到独立地址段，但与用户程序共享栈和地址空间、拥有独立堆空间。

入口点同时承担程序入口与 Syscall Dispatcher 双重功能

调用流程

1. 用户程序发起 Linux 系统调用
2. 内核判断调用号不以 0xFFFF 开头 → 为 Linux 兼容程序
3. 转发到 Linux Subsystem 处理
4. Subsystem 映射到 Capability 模型并返回结果

巧妙之处: 系统调用号以 0xFFFF 开头与本内核调用自然区分；t6 寄存器作为返回地址寄存器被覆盖，因其是临时寄存器，几乎不影响用户程序。长远方案: 重写 musl/glibc 彻底消除 t6 破坏问题。

等待线程计数器调度优化

核心思想: 在微内核设计中，提供服务的内核线程默认视为“善意线程”。一个被更多线程等待的服务线程，应当获得更多的调度机会以尽快完成服务、释放资源。为此，在每个 TCB 中维护一个等待计数器 waiter。

等待发生时 — **update(x)**

当线程 A 等待线程 B 时: $B.waiter += A.waiter$ ，并沿着等待链向上传播至链顶。时间复杂度 $O(d)$ ， d 为等待链深度

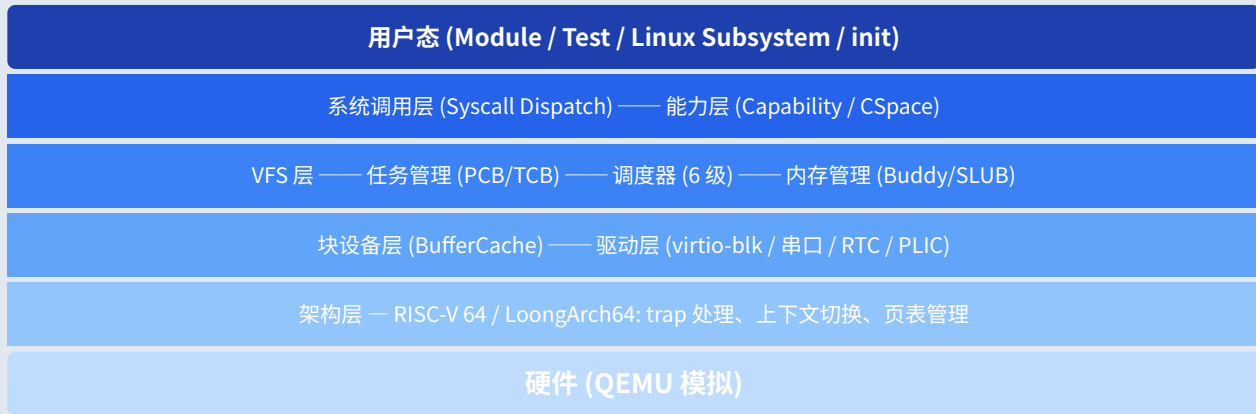
唤醒发生时 — **reduce(x)**

当线程 A 被唤醒时: $B.waiter -= A.waiter$ 。由于 A 被唤醒意味着 B 不再被 A 等待，无需向上传播。时间复杂度 $O(1)$

调度应用: RR 调度中时间片 = $N \times waiter$ ；CFS 调度中 $vruntime = runtime / waiter$

微内核中服务线程之间的等待关系通常较为简单，等待链深度 d 很小 $\rightarrow O(d) \approx O(1) \rightarrow$ 实际非常高效。

总体架构



内核态负责 VFS、内存管理、进程管理、
关键驱动，保证高效执行

同时支持 RISC-V 64 与 LoongArch64
双硬件架构

全部使用 C++ 实现，支持 C++26 反射
机制优化 RPC 胶水代码

混合内核设计哲学

Sustcore 将 VFS、内存管理、进程管理及关键驱动与文件系统收归内核态。相比纯微内核减少 IPC 开销提升效率，相比纯宏内核通过 Capability 机制保持强隔离性。

特性	微内核	Sustcore 混合内核
安全性	高（服务进程强隔离）	高（Capability 权限控制）
效率	较低（IPC 路径长、开销大）	较高（关键路径在内核态完成）
模块化	高（所有驱动在用户态）	中等（内核态+用户态分工）
可靠性	驱动崩溃不影响内核	关键驱动在内核，用户态驱动规划中
Linux 兼容	无（需改写所有程序）	通过 Linux 子系统透明兼容

设计权衡: 将高频、关键的操作路径放在内核态 → 高效率；将复杂但非关键的逻辑放在用户态 → 安全隔离。两类路径各取所长。

从零构建的技术栈

Sustcore 不基于任何现有内核，除 libfdt 与少量头文件外全部自实现。以下为核心自研组件：

- ▶ ext4 文件系统 — 完整读写支持，含 extent tree 与 BufferCache
- ▶ ELF 加载器 — 支持按需分页加载 (demand paging)
- ▶ SLUB 对象分配器 — 按 2 的幂大小分派，三链表管理
- ▶ Buddy 页框分配器 — MAX_ORDER=15，伙伴合并/分裂
- ▶ SV39 两级页表 — 同时支持 RISC-V 与 LoongArch64
- ▶ 自旋锁与同步原语
- ▶ PLIC / CLINT 中断控制器驱动
- ▶ virtio-blk 块设备驱动
- ▶ TarFS / DevFS / tmpfs / procfs 四种文件系统
- ▶ Endpoint IPC + librpc 远程过程调用框架
- ▶ FDT 设备树解析与统一 DeviceNode 抽象
- ▶ NS16550 串口 + RTC 时钟驱动
- ▶ 6 级优先级调度器 (RT/INIT/RR/FCFS/IDLE)
- ▶ 多平台 + 组件化构建系统 (RISC-V & LoongArch64)

总计自实现约 20+ 个核心子系统，覆盖架构支持、内存管理、文件系统、设备驱动、IPC/RPC、调度等操作系统全部关键领域。

内存管理子系统

三层物理内存分配架构

分配器	核心功能	分配单位
Buddy	管理空闲物理页框。MAX_ORDER=15，支持伙伴页的合并与分裂，减少外部碎片	4KB 页
SLUB	小对象分配器。按 2 的幂将请求分派至对应 FixedSizeAllocator slab，维护 empty/partial/full 三个链表	变长
GFP	统一分配入口。封装 Buddy 与 SLUB，叠加物理页引用计数，提供 Get Free Page 语义	整合层

虚拟内存管理: VMA 支持 CODE / DATA / STACK / HEAP / SHARE_RWX / SHARE_RO 等类型

► COW 写时复制 • 懒分配按需映射物理页 • 页缓存 + LRU 淘汰策略 • 文件映射 (file-backed memory)

Memory Capability 与 VMA 分离设计: 同一个 Memory 对象可以在不同进程中被映射到不同的虚拟地址，灵活且安全。

SLUB 分配器迁移：发现并修复 5 个内核 Bug

背景: 将内核全局分配器从 LinearGrowAllocator (LGA) 切换到 SLUB 后出现多种崩溃。根因在于 LGA 的 free() 是空操作，内存从不复用，系统性地掩盖了 use-after-free、double-free、链表损坏等内存安全 bug。SLUB 真正释放并复用内存，使这些 bug 立即暴露。

Bug	类型	根因与修复方案
#1	Double Free	EndpointPayload() 析构时同一 EndpointMessage 被 messages 链表和 pending_sends 链表各 delete 一次 → 先处理 pending_sends，将其关联消息从 messages 中摘除再 delete
#2	Use-after-free	PendingEndpointSend 的 cancel callback 通过 std::function 捕获裸指针，对象析构后 Future 仍可能触发 callback → 析构前先清空 callback 为 {}
#3	链表损坏	MixedSizeAllocator::init() 使用 placement new 直接覆盖旧对象而未调用析构函数 → 旧链表的 D_size 未清零、list_head 仍指向垃圾地址 → 先显式析构再 placement new
#4	nullptr 解引用	修复 #1、#2 后揭示的边界场景
#5	引用计数泄漏	修复后对引用计数逻辑做了进一步微调

关键启示: 一个看似“无害”的 no-op free() 可以系统性地掩盖大量内存安全 bug。SLUB 的正确实现不仅提升了内存利用率，更驱动了代码质量的全面提升。

VFS 虚拟文件系统

四层架构: Capability 层 (权限检查入口) → VFS 运行时层 (挂载表、inode 缓存、路径解析) → 文件系统接口层 (IFsDriver / ISuperblock / IINode 统一契约) → 具体实现

文件系统	类型	功能说明
TarFS	只读	解析 initrd 中的 ustar 格式 archive 文件, inode 编号 = tar header offset / 512
DevFS	伪 FS	挂载于 /sys/, 将内核设备暴露为文件节点。通过 CharFactory 延迟创建字符设备文件
ext4	完整读写	完整的 ext4 磁盘布局实现: superblock、block group descriptors、inode table、extent tree。使用 BufferCache 块级缓存
tmpfs	内存 FS	基于纯内存的临时文件系统, 继承 IPesudoFsDriver, 文件与目录内容全部保存于内存中, 适合 /tmp
procfs	伪 FS	挂载于 /proc/, 将内核中的进程、线程信息暴露为文件节点, 支持信息查询

支持 NONE / SHARED / PERMANENT 三种 inode 缓存策略 + BufferCache 块级缓存与 LRU 淘汰

进程与线程管理

PCB (进程控制块)

维护 PID、地址空间 (ASpace)、Capability Holder (CSpace)、线程链表 (TCB List) 和文件描述符表。fork 创建子进程时通过 CLONE/MIGRATE_ONCE 机制继承父进程 Capability

TCB (线程控制块)

维护 TID、256KB 内核栈、CPU 上下文 (Context)、调度信息 (优先级/时间片) 和等待信息 (等待计数器 waiter)。调度器以 TCB 为基本调度单元

核心系统调用:

- ▶ fork — 创建子进程, Capability 通过 CLONE/MIGRATE_ONCE 权限控制继承, CapIdx 在父子进程中保持一致
- ▶ execve — 解析 ELF64 文件, 为 PT_LOAD 段创建 Memory + VMA, 采用按需分配物理页策略
- ▶ Signal — 当前在内核态实现, 规划借助 TCB Suspend/Resume/Context 能力和 Fault/Notification 能力迁移到用户态

调度器：6 级优先级调度

调度类	优先级	调度算法	适用场景
RT	0 (最高)	实时 FIFO	对延迟敏感的实时任务
INIT	1	双槽位	启动阶段 kinit 与 init 内核线程
RR	2	时间片轮转	普通进程, <code>TIME_SLICES = 5</code>
FCFS	3	普通 FIFO	普通先来先服务
IDLE	4	空闲	最低优先级空闲线程, 无任务时运行
BOT	5	—	遍历下界标记, 非实际调度类

等待计数器集成: 在 RR 调度中时间片 = $N \times \text{waiter}$, 在 CFS 调度中 $\text{vruntime} = \text{runtime} / \text{waiter}$ 。占用资源多的服务线程因被更多线程等待, 自动获得更多调度权重, 从而更快完成服务、释放资源。

Endpoint IPC 与 RPC 框架

Endpoint 核心能力

- `endpoint_send / recv` — 同步消息传递
- `endpoint_send_async / recv_async` — 异步尝试
- `endpoint_call / reply` — 一次性请求-回复
- 随消息传递 Capability（能力授予）

Call / Reply 模型设计

调用方执行 `endpoint_call` 时, 内核自动创建一个临时的一次性 Reply Capability 随请求消息发送给服务端。服务端通过 `endpoint_reply` 返回结果后该 Capability 自动销毁, 生命周期清晰, 不会与其他请求混淆。

librpc 框架: 在 Endpoint 之上封装 `service_magic`(服务标识) + `function_id`(函数编号) 区分不同调用 • 参数编码/解码规则 • 支持 C++26 反射元编程自动生成胶水代码 (需 GCC 16+) • per-Session 对话模型 (规划中)

设备模型与驱动框架

设计目标: 驱动不直接读取 DTB/ACPI 原始结构, 统一通过 DeviceNode 接口查询属性, 实现平台无关。

组件	职责说明
DeviceModel	全局单例, 统一维护设备节点、物理内存区域 (FREE/RESERVED 等状态)、CPU 信息、中断管理器 (IrqManager) 和各 Provider
DeviceNode	平台无关的统一设备节点接口。当前支持 FDT 与 PCI 两种平台类型, 驱动层只依赖此接口而不依赖具体 FDT/PCI 结构
FDT Provider	DTB 解析后端, 将 Flattened Device Tree 中的所有硬件设备节点导入为 DeviceNode
DriverModel	管理驱动工厂注册、probe 设备匹配、create 创建运行时驱动对象, 并在 DevFS 中创建 /sys/ 目录项
MMIOResource	将 DeviceNode 中的 MMIO 声明转换为带内核虚拟地址映射 (KVA) 的运行时资源
VIrqResource	将中断声明转换为虚拟中断号 (virq), 提供 register_handler / enable / disable 接口

已实现驱动: NS16550 串口 • RTC (Goldfish for RISC-V / LS7A for LoongArch64) • PLIC + CLINT 中断控制器 • virtio-blk 块设备 • syscon-poweroff 电源管理 • PCI Host Bridge

构建系统

构建系统从零设计实现，支持 C/C++/ASM 多语言混合编译。顶层 Makefile 负责流程调度，script/ 下通用模板定义实际编译链接行为。

多平台支持

RISC-V 64 与 LoongArch64 双架构。通过交叉编译器前缀切换和 config.json 中每个架构独立的编译选项、feature 开关、QEMU 参数，实现一套源码多平台构建

组件化构建

静态库、内核模块、内核本体统一套用同一组件框架。每个组件通过 options.mk 描述配置、include.mk 声明源文件，通用模板负责具体的编译和链接逻辑

配置自动生成

从 config.json 自动生成 config.mk (Make 消费) 和 loggers.h (Logger 配置头文件) 等中间产物。支持命令行临时覆盖默认配置，避免 Makefile 硬编码

initrd 整合

将 module、init、linux-subsystem 等模块编译后整理进 initrd (tar 格式)，再通过 objcopy 转化为中间目标文件，最终链接到内核镜像的特定段中

使用界面: make build / make run / make dbg / make all — 简洁统一的命令入口

进程 Bootstrap 机制

问题: 进程启动时已由父进程通过 Capability 传递持有部分能力，但这些 Capability 本身不具有语义——进程无法知道某个 CapIdx 对应的是标准输入还是堆内存还是可执行文件。

方案: 在进程栈上额外加载一段 Bootstrap 信息，位于标准 ABI 的 argc/argv/envp/auxv 之后。

栈顶(sp) → argc / argv[] / nullptr → envp[] / nullptr → auxv[] / nullptr → bsargc → bsargv[] (每条指向一个 Bootstrap 条目)

支持的 Bootstrap 类型:

- ▶ TYPE_CAPEXP — 解释某个 CapIdx 的含义 (如 “#self” = 自身 PCB, “#cwd” = 工作目录, “#stdout” = 标准输出)
- ▶ TYPE_VADDREXP — 解释某个虚拟地址的含义 (如 “#heap” = 堆起始地址)
- ▶ TYPE_PATHEXP — 解释某个路径的含义 (如 “#cwd:/home” = 当前工作目录绝对路径)
- ▶ TYPE_SHELLIO — 解释 Shell I/O 的重定向模式 (覆盖 or 追加) 和目标 (stdin/stdout/stderr)

以 “#” 开头的描述字符串具有内核预定义的语义，用户也可以自定义其他描述字符串，内核不做限制。

测试体系（28 个测试模块）

IPC / RPC 测试

- ▶ test_endpoint_master / slave — IPC 端点通信
- ▶ test_call_service / user — 服务调用
- ▶ test_rpc_client / server — RPC 框架

文件系统测试

- ▶ test_ext4_create / read / rw / permission / symlink
- ▶ test_file_rw_a / b — 双进程文件读写
- ▶ test_fs_score — 文件系统评分测试

进程 / 线程 / 信号 / ELF

- ▶ test_thread • test_fork • test_execve • test_signal
- ▶ test-elf-demand • test-elf-demand-perf • test-elf-demand-perf-child

内存与系统测试

- ▶ test_file_backed_memory — 文件映射内存
- ▶ test_page_cache / test_page_cache_perf — 页缓存
- ▶ test_meminfo • test_procfs • test-linux
- ▶ contest-runner — 评测运行器

关键技术验证

一、SLUB 分配器迁移中的 5 个 Bug 修复

- ▶ Bug #1: 同一 EndpointMessage 指针被 messages 链表和 pending_sends 链表各 delete 一次 → 先处理 pending_sends, 将其关联消息从 messages 中摘除后再 delete
- ▶ Bug #2: std::function cancel callback 捕获裸指针, 对象析构后 Future 仍可能触发 → 析构前将 callback 清空为 {}
- ▶ Bug #3: MixedSizeAllocator::init() 用 placement new 直接覆盖旧对象, 未调析构导致 alloc_records 链表损坏 → 先显式调用析构函数, 再 placement new 重建

LGA 的 no-op free() 系统性地掩盖了 use-after-free、double-free、链表损坏等内存安全 bug。SLUB 的实现不仅提升了内存利用率, 更驱动了代码质量的全面提升。

二、QEMU stvec 寄存器处理 Bug 的发现与溯源

- ▶ 发现问题: 启用 RISC-V vectored 中断模式后, 中断无法正常进入 ISR, pc 寄存器值错误地等于 stvec 本身
- ▶ 深入 QEMU 源码溯源: 旧版本 QEMU 在 trap 处理时直接执行 `env->pc = env->stvec`, 未对 MODE 字段做 `>> 2 << 2` 处理
- ▶ 该修复于 2019 年提交 (commit acbbb94e), 但直到 QEMU 10.1.0 版本才被合并进 master 分支

展示了深入第三方工具链源码进行根因分析的能力, 以及将理论与实践对照验证的工程素养。

待完善之处

短期改进计划

多线程	完成 PCB/TCB 完整能力设计, 实现 TCB Suspend/Resume/Context 接口, 支持多核 IPI 挂起
用户态信号	借助 Fault 能力 (同步信号) 和 Notification 能力 (异步信号) 将信号处理逻辑从内核态迁移到用户态, 实现 sigframe 创建与 sigreturn
symlink 安全	修复符号链接可以绕过父目录 Capability 权限检查的设计缺陷, 保证基于目录的访问控制完整性
文档补充	为 ext4 完整实现、procfs、tmpfs 以及各具体硬件驱动补写独立详细的设计与实现文档

中长期发展规划

用户态驱动	实现网络协议栈、文件系统驱动等复杂子系统在用户态进程中运行, 发挥微内核的隔离优势
Session 化 RPC	将 librpc 从单次 call/reply 扩展为 per-Session 对话模型, 支持"建立会话 → 多次调用 → 关闭会话"的完整生命周期
00P 用户态接口	在 libc (kmodlibc / linuxss-libc) 中对系统调用再封装一层 00P 接口, 将 CapIdx 内化为对象的唯一成员, 使代码更语义化
musl/glibc 适配	重写 libc 添加兼容层, 将所有 Linux 系统调用改为调用兼容层内部函数, 彻底消除 t6 寄存器破坏问题
PCI 枚举完善	完善 PCI 总线设备的自动枚举、配置空间读取、BAR 解析与驱动自动匹配

总结与致谢

Capability 安全模型

双层 CSpace 架构
Payload + Permission 分离
权限传递链与引用计数
贯彻最小权限原则

从零构建技术栈

自实现 ext4 / SLUB / ELF
RISC-V + LoongArch 双架构
20+ 核心子系统原创实现
支持 C++26 反射元编程

Linux 子系统兼容

用户态兼容层设计
共享地址空间透明转发
无需修改 musl/glibc
28 个模块全面测试

参考项目与致谢

seL4 — 灵感来源与系统调用设计依托

Linux · NAOS · Managarm · RISC-V / LoongArch 官方文档

Tayhuang OS · Orange' S · OS: Three Easy Pieces

感谢阅读！

github.com/sustcore-team/sustcore